



Department of Electronics & Communications

25SC2107E – Machine Learning

Skilling Experiment

California Housing — Linear Regression and Regularisation

Dataset: California Housing | Course Outcome: CO2 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Hyderabad

1 Experiment : California Housing — Linear Regression and Regularisation

Dataset: California Housing

Course Outcome: CO2

1.1 Aim

To build a model that predicts California house prices, to build the same model again from scratch using NumPy gradient descent, and then to compare three ways of keeping a model simple: Ridge, Lasso and ElasticNet.

1.2 Learning Objectives

After completing this experiment, you should be able to:

- Say how a *regression* problem is different from the classification problems in Experiments 1 to 3.
- Fit `LinearRegression` and check how good it is using RMSE and R^2 .
- Write gradient descent yourself in NumPy and confirm it gives the same answer as scikit-learn.
- Explain what Ridge, Lasso and ElasticNet penalise, and what `alpha` does.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib scikit-learn
```

1.4 Dataset

California Housing, taken from the 1990 US census. Each row is one census *district* (a small area), not a single house.

- 20,640 rows, 8 number columns, no missing values. So there is no cleaning work to do.
- **Answer column:** `MedHouseVal`, measured in hundreds of thousands of dollars. So 2.5 means \$250,000.
- It is downloaded by `fetch_california_housing` (about 400 kB). It is saved on your computer afterwards, so it works offline next time.

1.5 Procedure

1. Create a notebook `skill1_lab4.ipynb`;
2. Load California Housing and look at the answer column.
3. Split the data into training and test sets, then scale the features.
4. Fit `LinearRegression` and report RMSE and R^2 .
5. Do the same fit by hand with NumPy and plot the loss curve.
6. Compare the two sets of coefficients.
7. Plot test RMSE against `alpha` for Ridge, then for Lasso.
8. Put all three sets of coefficients on one bar chart.
9. Write `regression_benchmark()` to rank every model by its test error.

1.6 Notebook Implementation

Cell 1 — Import the libraries

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.datasets import fetch_california_housing
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.linear_model import LinearRegression, Ridge, Lasso,
   ElasticNet
8 from sklearn.metrics import mean_squared_error, r2_score
```

- The four models on the `linear_model` line are much more similar than their names suggest. All four draw a straight line. The only difference is how strongly they are penalised for using big coefficients.

Cell 2 — Load the data

```
1 data = fetch_california_housing(as_frame=True)
2 housing = data.frame
3 print(housing.shape)
4 housing.head()
```

- `as_frame=True` gives you a pandas table, so the columns keep their names. You will need those names in Cell 10.
- `data.frame` keeps the 8 features and the answer in one table: 20,640 rows and 9 columns.

Cell 3 — Look at the answer column

```
1 plt.hist(housing["MedHouseVal"], bins=50)
2 plt.title("Median House Value")
3 plt.show()
```

- The shape is right-skewed: most districts are cheap, and a long thin tail of expensive districts stretches out to the right.
- Look at the far right. There is a tall thin spike exactly at 5.0. This happened because the data was capped. Every district worth more than \$500,000 was simply written down as 5.0. So those districts are not all worth the same amount — we only know they are worth at least that much.
- What this means: the model can never predict a value above 5.0, so its worst mistakes will be on the most expensive districts.

Cell 4 — Split into training and test sets

```
1 X = housing.drop(columns="MedHouseVal")
2 y = housing["MedHouseVal"]
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.2, random_state=42)
6 print(X_train.shape, X_test.shape)
```

- 16,512 rows are used to train the model, and 4,128 rows are kept aside to test it.

Cell 5 — Scale the features

```
1 scaler = StandardScaler()
2 X_train_s = scaler.fit_transform(X_train)
```

```

3 X_test_s = scaler.transform(X_test)
4 print(X_train_s.mean().round(3), X_train_s.std().round(3))

```

Fit the scaler on the training data, then transform both sets. Here scaling is compulsory, for two separate reasons:

- Gradient descent (Cell 6) would not work. It uses one step size for all features at the same time. Population is in the thousands while AveBedrms is close to 1, so no single step size can suit both.
- Regularisation (Cell 8 onwards) would be unfair. Ridge and Lasso penalise large coefficients. A feature measured in thousands naturally gets a very small coefficient, so it would escape the penalty for no real reason.

Cell 6 — Fit with scikit-learn, then by hand

```

1 lr = LinearRegression()
2 lr.fit(X_train_s, y_train)
3 pred = lr.predict(X_test_s)
4 print("RMSE:", round(np.sqrt(mean_squared_error(y_test, pred)), 4))
5 print("R2  :", round(r2_score(y_test, pred), 4))
6
7 w = np.zeros(X_train_s.shape[1])
8 b = 0.0
9 rate = 0.1
10 losses = []
11
12 for step in range(500):
13     pred_train = X_train_s @ w + b
14     error = pred_train - y_train.to_numpy()
15     losses.append((error ** 2).mean())
16     w = w - rate * (X_train_s.T @ error) / len(y_train)
17     b = b - rate * error.mean()
18
19 print("Final MSE:", round(losses[-1], 4))

```

- **RMSE** ≈ 0.75 means the model is usually wrong by about \$75,000. $R^2 \approx 0.58$ means it explains a bit less than three fifths of the variation in prices. R^2 can even go *below zero*, which means the model is worse than just always guessing the average price. You will see this happen in the deliverable.
- The loop is the actual training algorithm that `lr.fit()` was doing for you behind the scenes. `w` stores one weight for each feature and `b` is the intercept; both start at zero. `X @ w + b` makes predictions for all 16,512 rows in one go.
- **rate** is the learning rate, that is, the step size. If it is too small, 500 steps will not be enough. If it is too large, the loop overshoots and you get NaN.

Cell 7 — The loss curve, and did it match?

```

1 plt.plot(losses)
2 plt.xlabel("step")
3 plt.ylabel("mean squared error")
4 plt.title("Gradient Descent Loss")
5 plt.show()
6
7 compare = pd.DataFrame({
8     "feature": X.columns,
9     "gradient_descent": w.round(4),
10    "sklearn": lr.coef_.round(4),
11 })

```

12 `compare`

- The curve should drop quickly and then go flat. Flat means the model has converged, that is, it has finished learning. If it is still falling at step 500, the learning rate is too small. If it goes up or becomes NaN, the rate is too large. If it is flat right from the start, you forgot to scale the features.
- The two coefficient columns match to four decimal places. **So LinearRegression is not magic** — you have just rebuilt it in eleven lines. Scikit-learn solves the equations directly in one exact step, while gradient descent walks downhill in many small steps. The advantage of gradient descent is that it still works when the data is too big to solve in one go.
- **MedInc** has by far the biggest weight: the income of a district matters more than anything else for house price.

Cell 8 — Ridge: test error against alpha

```

1 alphas = [0.001, 0.01, 0.1, 1, 10, 100]
2 ridge_rmse = []
3
4 for a in alphas:
5     model = Ridge(alpha=a)
6     model.fit(X_train_s, y_train)
7     ridge_rmse.append(np.sqrt(mean_squared_error(y_test,
8         model.predict(X_test_s))))
9
10 plt.plot(alphas, ridge_rmse, marker="o")
11 plt.xscale("log")
12 plt.xlabel("alpha")
13 plt.ylabel("test RMSE")
14 plt.title("Ridge")
15 plt.show()

```

- **Ridge is just linear regression with a penalty added** for the *squared* size of each coefficient. Think of the penalty as a fee the model pays every time it uses a big weight. **alpha** decides how big that fee is. At 0 it is ordinary linear regression. As **alpha** grows, all the coefficients get smaller and smaller, but **none of them ever becomes exactly zero**.
- `yscale("log")` is needed because the alpha values cover five powers of ten.
- The curve is almost flat. That is the correct answer, not a mistake: with 16,512 rows and only 8 features there is very little overfitting to fix. **“It made no difference” is itself a valid result worth reporting.**

Cell 9 — Lasso: test error against alpha

```

1 lasso_rmse = []
2
3 for a in alphas:
4     model = Lasso(alpha=a)
5     model.fit(X_train_s, y_train)
6     lasso_rmse.append(np.sqrt(mean_squared_error(y_test,
7         model.predict(X_test_s))))
8
9 plt.plot(alphas, lasso_rmse, marker="o")
10 plt.xscale("log")
11 plt.xlabel("alpha")
12 plt.ylabel("test RMSE")
13 plt.title("Lasso")
14 plt.show()

```

- **Lasso penalises the *plain* size of each coefficient instead of the squared size.** That one small change means a coefficient can become **exactly zero**, which removes that feature completely. So Lasso picks features for you while it trains.
- Ridge stayed flat, but Lasso stays flat and then shoots up. By $\alpha = 1$ every coefficient is zero, so the model just predicts the same number every time, and the RMSE becomes the same as simply guessing the average.
- **Too much regularisation does not just stop helping. It ruins the model completely.**

Cell 10 — What the penalties did to the coefficients

```

1 coefs = pd.DataFrame({
2     "LinearRegression": lr.coef_,
3     "Ridge a=1": Ridge(alpha=1).fit(X_train_s, y_train).coef_,
4     "Lasso a=0.1": Lasso(alpha=0.1).fit(X_train_s, y_train).coef_,
5 }, index=X.columns)
6
7 coefs.plot.bar(figsize=(10, 4))
8 plt.ylabel("coefficient")
9 plt.title("Coefficients under each penalty")
10 plt.show()

```

- The Ridge bars are slightly shorter than the plain regression bars: they have shrunk, but every one is still there. The Lasso bars are different: several have dropped all the way to exactly zero.
- **Ridge says: keep every feature, but use each one gently. Lasso says: use fewer features.**
- MedInc survives every penalty. When a feature refuses to be regularised away, that is a strong hint that it really matters.

Deliverable Function

```

1 def regression_benchmark(X_train, y_train, X_test, y_test, alphas):
2     models = {"LinearRegression": LinearRegression()}
3
4     for a in alphas:
5         models["Ridge a=" + str(a)] = Ridge(alpha=a)
6         models["Lasso a=" + str(a)] = Lasso(alpha=a)
7         models["ElasticNet a=" + str(a)] = ElasticNet(alpha=a,
8             l1_ratio=0.5)
9
10    rows = []
11    for name in models:
12        model = models[name]
13        model.fit(X_train, y_train)
14        pred = model.predict(X_test)
15        rows.append({
16            "model": name,
17            "test_rmse": np.sqrt(mean_squared_error(y_test, pred)),
18            "test_r2": r2_score(y_test, pred),
19            "zero_coefs": int((model.coef_ == 0).sum()),
20        })
21
22    table = pd.DataFrame(rows)
23    table = table.sort_values("test_rmse").reset_index(drop=True)
24    return table.round(4)

```

```

25
26 results = regression_benchmark(X_train_s, y_train, X_test_s, y_test,
27                               [0.01, 0.1, 1, 10])
28 results

```

- This pattern is worth remembering: make a dictionary of named models, loop over it, save one dictionary of results per model, and turn the whole list into a DataFrame at the end. If you want to add a fifth model later, it costs you one line.
- **ElasticNet** is a mixture of the other two. `l1_ratio=0.5` means half Lasso and half Ridge, so it can delete a coefficient and still cope well when features are related to each other.
- **Read zero_coefs together with RMSE.** It stays at 0 for every Ridge model, and rises for Lasso and ElasticNet as `alpha` grows. At the bottom of the table it reaches 8, meaning every feature is gone. Those rows have $R^2 \approx 0$, which is the tell-tale sign of a model that has been penalised so hard it now predicts one fixed number forever.
- Sorting by `test_rmse` puts the best model at the top. Expect the top few rows to be almost equal: on this dataset regularisation does not help or hurt much, and a table that shows those ties honestly is more useful than one that hides them.

1.7 Expected Output

```

(20640, 9)
(16512, 8) (4128, 8)
RMSE: 0.7456
R2   : 0.5758
Final MSE: 0.5179

      model  test_rmse  test_r2  zero_coefs
0  LinearRegression    0.7456    0.5758         0
1      Ridge a=0.01    0.7456    0.5758         0
2      Ridge a=0.1    0.7456    0.5758         0
3      Ridge a=1    0.7456    0.5758         0
...
11      Lasso a=1    1.1453   -0.0001         8
12  ElasticNet a=10    1.1453   -0.0001         8

```

1.8 Result

Linear regression on the scaled California Housing features gave a test RMSE of about 0.75, which is roughly \$75,000, and an R^2 of about 0.58. The hand-written NumPy gradient descent, run for 500 steps with a learning rate of 0.1, ended up with coefficients that match scikit-learn's answer to four decimal places. Ridge left the error almost unchanged. Lasso and ElasticNet slowly removed more and more coefficients until all 8 were zero, at which point the model could only predict a single constant value.

Note

The three penalties, one line each

- **Ridge** penalises *squared* coefficients: it shrinks them all but deletes none. Use it when every feature is a little bit useful.
- **Lasso** penalises *plain* coefficients: it can set them to exactly zero. Use it when you think many features are useless and you want the model to tell you which ones.
- **ElasticNet** is a mix of both, controlled by `l1_ratio`. Use it when features are related to each other *and* you still want feature selection.

In all three, `alpha` works like a volume knob. Turn it down to zero and you are back to plain linear regression. Turn it up far enough and the model just predicts the same number forever.